

JavaScript Base — Appunti

1. Variabili: let e const

In JavaScript esistono due modi per creare una variabile. "let" crea una variabile che si può modificare in seguito. "const" crea una costante: il valore non si può più riassegnare. Regola pratica: usa sempre const di default, passa a let solo quando sai già che quel valore dovrà cambiare (es. un contatore, un punteggio).

```
let eta = 16;
eta = 17; // va bene, "let" si può riassegnare

const nomeScuola = "Liceo Italia";
// nomeScuola = "Altro Liceo"; // ERRORE! const non si riassegna
```

2. Tipi di dato: stringa, numero, booleano, undefined, null

string è testo, number è un numero (intero o decimale, senza distinzione), boolean è true o false, undefined è una variabile dichiarata ma senza valore assegnato, null è un valore vuoto assegnato di proposito da chi scrive il codice.

```
const stringa = "Ciao mondo";
const numero = 42;
const booleano = true;
let nonDefinita; // undefined
const valoreNullo = null; // vuoto, ma di proposito

console.log(typeof stringa); // "string"
console.log(typeof numero); // "number"
```

3. Operatori aritmetici, di confronto, logici

Gli operatori aritmetici (+ - * /) servono per fare calcoli. Gli operatori di confronto (===, !==, >, <) confrontano due valori e restituiscono true o false: usa sempre === e !== (confronto forte) invece di == e != per evitare conversioni automatiche impreviste. Gli operatori logici combinano condizioni: && (e) è vero solo se entrambe le condizioni sono vere, || (o) è vero se almeno una lo è, ! (non) inverte true/false.

```
const a = 10, b = 3;
console.log(a + b); // 13
console.log(a / b); // 3.333...

console.log(5 === 5); // true
console.log(5 === '5'); // false, tipi diversi

const haStudiato = true, haDormitoBene = false;
console.log(haStudiato && haDormitoBene); // false
console.log(haStudiato || haDormitoBene); // true
```

4. if / else

Permette di eseguire codice diverso a seconda che una condizione sia vera o falsa. Esempio pratico: controllare se un voto è sufficiente oppure no.

```
const voto = 5.5;

if (voto >= 6) {
  console.log(`Voto ${voto}: sufficiente!`);
} else {
  console.log(`Voto ${voto}: insufficiente.`);
}
```

5. for classico

Il ciclo for ripete un blocco di codice un numero definito di volte. Ha tre parti: inizializzazione, condizione di continuazione, aggiornamento ad ogni giro.

```
for (let i = 1; i <= 5; i++) {
  console.log("Numero:", i);
}
// stampa i numeri da 1 a 5
```

6. while

Il ciclo while ripete un blocco di codice finché una condizione resta vera. A differenza del for, non sappiamo in anticipo quante volte girerà.

```
let benzina = 5; // litri rimasti

while (benzina > 0) {
  console.log("Benzina rimasta:", benzina);
  benzina = benzina - 1;
}
console.log("Serbatoio vuoto!");
```

7. Funzioni: modo classico e modo freccia

Una funzione è un blocco di codice riutilizzabile a cui possiamo passare dati (parametri) e che può restituire un risultato (return). In JavaScript moderno si usano molto le arrow function, una sintassi più corta e diretta.

```
// Modo classico
function saluta(nome) {
  return `Ciao ${nome}, benvenuto!`;
}

// Modo freccia
const salutaFreccia = (nome) => {
  return `Ciao ${nome}, benvenuto!`;
};

// Freccia compatta (una riga)
const salutaFrecciaCorta = (nome) => `Ciao ${nome}, benvenuto!`;
```

8. Array: cos'è, come si crea, come si accede

Un array è una lista ordinata di valori. Ogni elemento ha una posizione (indice) che parte da 0.

```
const frutti = ["mela", "banana", "kiwi", "arancia"];

console.log(frutti[0]); // "mela"
console.log(frutti[2]); // "kiwi"
console.log(frutti.length); // 4

frutti[1] = "pera"; // modifica un elemento
```

9. Metodi array: forEach

forEach esegue un'azione per ogni elemento dell'array. Non restituisce un nuovo array, serve solo per "fare qualcosa" con ogni valore.

```
const numeri = [5, 12, 8, 3, 20, 1];

numeri.forEach((n) => {
  console.log("Elemento:", n);
});
```

10. Metodi array: map

map applica una trasformazione a ogni elemento e crea un NUOVO array con i risultati, senza modificare l'originale.

```
const numeri = [5, 12, 8, 3, 20, 1];
const raddoppiati = numeri.map((n) => n * 2);
console.log(raddoppiati); // [10, 24, 16, 6, 40, 2]
```

11. Metodi array: filter

filter crea un NUOVO array con solo gli elementi che rispettano una condizione (predicato).

```
const numeri = [5, 12, 8, 3, 20, 1];
const maggioriDi5 = numeri.filter((n) => n > 5);
console.log(maggioriDi5); // [12, 8, 20]
```

12. Metodi array: find

find restituisce il PRIMO elemento che rispetta una condizione: un singolo valore, non un array.

```
const numeri = [5, 12, 8, 3, 20, 1];
const primoMaggioreDi10 = numeri.find((n) => n > 10);
console.log(primoMaggioreDi10); // 12
```

13. Metodi array: sort

sort ordina l'array. Di default ordina come testo, quindi per ordinare numeri serve una funzione di confronto.

```
const numeri = [1, 20, 3, 12];
const ordinati = [...numeri].sort((x, y) => x - y);
console.log(ordinati); // [1, 3, 12, 20]
```

14. Metodi array: reduce

reduce "riduce" tutto l'array a un unico valore, accumulando un risultato passo dopo passo (es. somma totale, media).

```
const numeri = [5, 12, 8, 3, 20, 1];
const somma = numeri.reduce((acc, n) => acc + n, 0);
console.log(somma); // 49
```

15. Dizionari (oggetti): cos'è, come si crea, come si legge

Un dizionario (in JS si chiama oggetto) raccoglie dati in coppie chiave: valore. Si legge sia con la notazione a punto sia con le parentesi quadre (utile quando la chiave è dentro una variabile).

```
const studente = { nome: "Luca", cognome: "Verdi", eta: 16 };

console.log(studente.nome);      // "Luca" (con il punto)
console.log(studente["cognome"]); // "Verdi" (parentesi quadre)

const chiave = "eta";
console.log(studente[chave]); // 16
```

16. for...in per scorrere un dizionario

Il ciclo for...in scorre tutte le chiavi di un oggetto, una per una, permettendo di leggerne anche il valore.

```
const studente = { nome: "Luca", cognome: "Verdi", eta: 16 };

for (const chiave in studente) {
  console.log(chiave, "->", studente[chave]);
}
```

17. Object.keys() e Object.values()

Object.keys(oggetto) restituisce un array con tutte le chiavi. Object.values(oggetto) restituisce un array con tutti i valori.

```
const studente = { nome: "Luca", cognome: "Verdi", eta: 16 };

console.log(Object.keys(studente)); // ["nome", "cognome", "eta"]
console.log(Object.values(studente)); // ["Luca", "Verdi", 16]
```

18. Array di dizionari (esempio: lista studenti)

Combinando array e dizionari possiamo rappresentare dati tabellari, come un registro di classe: ogni studente è un dizionario, la lista di studenti è un array.

```
const classe = [
  { nome: "Luca", voto: 7 },
  { nome: "Giulia", voto: 9 },
  { nome: "Marco", voto: 5 },
];

classe.forEach((s) => {
  console.log(`${s.nome} ha preso ${s.voto}`);
});

const promossi = classe.filter((s) => s.voto >= 6);
```

```
console.log(promossi);
```